

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 09 | Doubly Linked Lists, Circular Linked Lists and Their Applications

Week 9 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 09 at a glance

Item	Details
Lab title	Doubly Linked Lists, Circular Linked Lists and Their Applications
Week	9 of 16
Prerequisites	Lab 08 (Singly Linked List)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab09_Instructor.ipynb , Lab09_Student.ipynb
Next lab	Lab 10: Stacks

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the structure of a **doubly linked list** (prev and next references) and its advantages.
2. Implement a doubly linked list with **sentinel (dummy) head and tail** nodes.
3. Insert and delete at both ends and **delete a node in $O(1)$** when you have a reference to it.
4. Traverse a doubly linked list forwards and backwards.
5. Implement a **circular linked list** and traverse it safely.
6. Solve the **Josephus problem** and simulate **round-robin scheduling** with a circular list.
7. Build real applications: **browser history**, **music playlist** and an **LRU cache**.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement doubly and circular linked lists	Apply (C3)
CLO-3	Select an appropriate list variant for an application	Analyse (C4)

2. Motivation: Going Back and Going Round

A singly linked list can only move **forward**. You cannot go back to the previous node, and deleting the last node takes $O(n)$. Many real systems need to move **both ways** (back and forward in a browser) or **go round in circles** (players taking turns). Doubly and circular linked lists solve exactly these problems.

2.1 In Daily Life

Application	Structure	Why
Browser Back / Forward buttons	Doubly linked list	Move to previous and next page
Music player: previous / next song, repeat all	Circular doubly linked list	Both directions and wrap around
Image gallery on your phone	Doubly linked list	Swipe left and right
Ludo / card game turns	Circular linked list	After the last player, the first plays again

Application	Structure	Why
Ctrl+Z / Ctrl+Y (undo / redo)	Doubly linked list (or two stacks)	Step back and forward through edits
Traffic signal cycle (red → green → yellow → red)	Circular list	Repeats forever

2.2 In Industry

- **LRU (Least Recently Used) caches** in web browsers, databases, Redis and operating systems use a **hash map + doubly linked list** to evict old data in $O(1)$.
- **Operating systems** use circular queues of processes for **round-robin CPU scheduling**.
- **Text editors** store lines in doubly linked lists so the cursor can move up and down efficiently.
- Java's `LinkedList` and Python's `collections.deque` are implemented with doubly linked structures.

2.3 In Research

- **Cache replacement policies** (LRU, LFU, ARC) are actively researched for CDNs, databases and AI model serving (e.g., caching attention keys/values in large language models).
- **Real-time systems and networking** use circular buffers to process continuous sensor and network streams.

2.4 Why It Matters in 2026–2030

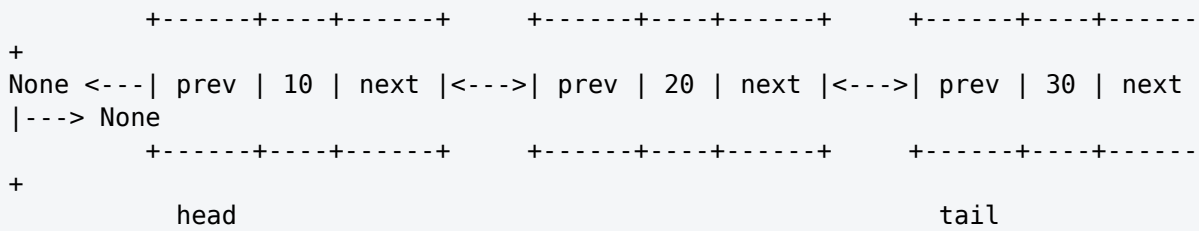
Caching is everywhere

Every fast app you use, from YouTube to ChatGPT, relies on **caching**: keeping recently used data close so it can be served instantly. The LRU cache you build today (Task 7) is the exact design asked in interviews at top companies and used in production systems.

3. Part A: Doubly Linked List

In a **doubly linked list (DLL)**, each node has **three parts**: **prev** (reference to the previous node), **data**, and **next**.

A doubly linked list



	Singly linked list	Doubly linked list
References per node	1 (next)	2 (prev, next)
Traverse backwards	Not possible	Yes
Delete last node	O(n)	O(1)
Delete a given node (with reference)	O(n), need previous node	O(1)
Memory	Less	More (one extra reference per node)
Code complexity	Simpler	More pointers to update

3.1 Sentinel Nodes: A Professional Trick

Handling empty lists and the head/tail special cases makes linked-list code messy. Professionals add two **dummy (sentinel) nodes** that never hold real data: **head** sentinel at the start and **tail** sentinel at the end. Every real node is always **between two nodes**, so insertion and deletion have **no special cases**.

DLL with sentinels

```
[HEAD] <-> [10] <-> [20] <-> [30] <-> [TAIL]      (HEAD and TAIL hold no data)
Empty list: [HEAD] <-> [TAIL]
```

```

class DNode:
    def __init__(self, data=None):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = DNode()          # sentinel
        self.tail = DNode()          # sentinel
        self.head.next = self.tail
        self.tail.prev = self.head
  
```

```

        self.size = 0

    def _insert_between(self, data, before, after):
        node = DNode(data)
        node.prev, node.next = before, after
        before.next = node
        after.prev = node
        self.size += 1
        return node

    def _unlink(self, node):          # 0(1) delete of a known node
        node.prev.next = node.next
        node.next.prev = node.prev
        self.size -= 1
        return node.data

    def push_front(self, data):
        return self._insert_between(data, self.head, self.head.next)

    def push_back(self, data):
        return self._insert_between(data, self.tail.prev, self.tail)

    def pop_front(self):
        if self.size == 0:
            raise IndexError("empty list")
        return self._unlink(self.head.next)

    def pop_back(self):
        if self.size == 0:
            raise IndexError("empty list")
        return self._unlink(self.tail.prev)

    def forward(self):
        out, cur = [], self.head.next
        while cur is not self.tail:
            out.append(cur.data); cur = cur.next
        return out

    def backward(self):
        out, cur = [], self.tail.prev
        while cur is not self.head:
            out.append(cur.data); cur = cur.prev
        return out

d = DoublyLinkedList()
for x in [20, 30, 40]:
    d.push_back(x)
d.push_front(10)
print("forward :", d.forward())
print("backward:", d.backward())
print("pop_back ->", d.pop_back(), "| pop_front ->", d.pop_front(), "|",
      d.forward())

```

3.2 Inserting in the Middle: Four Links

To insert node N between nodes A and B, **four** references change: **N.prev = A**, **N.next = B**, **A.next = N**, **B.prev = N**. Set the new node's links first, then update the neighbours.

```
Before:  [A] <-> [B]
1. N.prev = A      2. N.next = B
3. A.next = N      4. B.prev = N
After:   [A] <-> [N] <-> [B]
```

3.3 Deleting a Node in O(1)

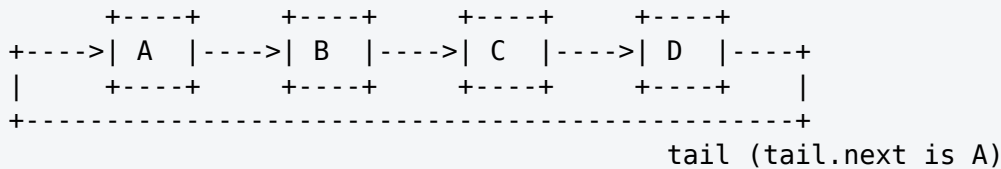
If we hold a reference to a node (for example from a dictionary), we can remove it instantly: its previous node skips forward over it and its next node skips backward over it. This is impossible in O(1) with a singly linked list, and it is the key to the LRU cache.

```
d2 = DoublyLinkedList()
handles = {name: d2.push_back(name) for name in ["Ali", "Sara", "Hamza",
"Zoya"]}
d2._unlink(handles["Hamza"])          # O(1): no searching
print(d2.forward())
```

4. Part B: Circular Linked List

In a **circular linked list**, the last node points back to the **first** node instead of None. There is no end, so you can keep going round. It is convenient to keep a reference to the **tail**: then **tail.next** is the head, giving $O(1)$ access to both ends.

Circular singly linked list



! Careful with traversal

`while cur is not None` loops **forever** on a circular list. Instead, stop when you come back to the starting node.

```

class CNode:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularList:
    def __init__(self):
        self.tail = None
        self.size = 0

    def append(self, data):
        node = CNode(data)
        if self.tail is None:
            node.next = node                # single node points to itself
        else:
            node.next = self.tail.next      # new node -> head
            self.tail.next = node           # old tail -> new node
        self.tail = node
        self.size += 1

    def to_list(self):
        if self.tail is None:
            return []
        out, cur = [], self.tail.next      # start at head
        while True:
            out.append(cur.data)
            cur = cur.next
            if cur is self.tail.next:       # back at the start
                break
        return out

    def rotate(self):                       # move head to the end in O(1)
        if self.tail:
            self.tail = self.tail.next

players = CircularList()
for p in ["Red", "Green", "Yellow", "Blue"]:
    players.append(p)
turn = players.tail.next
for _ in range(6):                        # six turns of Ludo
    print("Turn:", turn.data)
    turn = turn.next

```

4.1 Round-Robin Scheduling

An operating system gives each process a fixed **time slice** (quantum). If a process is not finished, it goes to the back of the circle. This is fair: no process waits forever.

```

def round_robin(jobs, quantum):
    """jobs: list of (name, burst_time). Returns finishing order and
    times."""
    ring = CircularList()
    for name, burst in jobs:
        ring.append([name, burst])
    time, finished = 0, []
    prev, cur = ring.tail, ring.tail.next
    while ring.size:
        run = min(quantum, cur.data[1])
        time += run
        cur.data[1] -= run
        if cur.data[1] == 0:                # job done: remove it from
the ring
            finished.append((cur.data[0], time))
            ring.size -= 1
            if ring.size == 0:
                break
            prev.next = cur.next
            if cur is ring.tail:
                ring.tail = prev
            cur = cur.next
        else:
            prev, cur = cur, cur.next
    return finished

print(round_robin([("P1", 5), ("P2", 3), ("P3", 8)], quantum=2))

```

5. Part C: Choosing the Right List

Need	Best choice
Simple stack or forward-only sequence	Singly linked list
Move both ways, delete known nodes in $O(1)$	Doubly linked list
Repeat in a cycle (turns, playlists, buffers)	Circular linked list
Both directions + cycle (repeat-all playlist)	Circular doubly linked list
Fast access by index	Array / Python list (not a linked list!)
Queue with fast operations at both ends	<code>collections.deque</code> (built on a doubly linked structure)

Python's built-in deque

```

from collections import deque
history = deque(maxlen=3) # keeps only the last 3 items
for page in ["google.com", "uo.edu.pk", "github.com", "leetcode.com"]:
    history.append(page)
print(history)
history.appendleft("start"); print(history.pop(), history)

```

6. C++ Corner

C++

```

#include <iostream>
#include <list>
using namespace std;

struct DNode {
    int data; DNode* prev; DNode* next;
    DNode(int d) : data(d), prev(nullptr), next(nullptr) {}
};

int main() {
    // Manual doubly linked list: 10 <-> 20 <-> 30
    DNode* a = new DNode(10); DNode* b = new DNode(20); DNode* c = new
DNode(30);
    a->next = b; b->prev = a; b->next = c; c->prev = b;
    for (DNode* p = c; p; p = p->prev) cout << p->data << " "; // backwards
    cout << endl;
    // delete b in O(1)
    b->prev->next = b->next; b->next->prev = b->prev; delete b;
    delete a; delete c;

    // The standard library doubly linked list
    list<string> playlist = {"Song A", "Song B", "Song C"};
    playlist.push_front("Intro");
    playlist.pop_back();
    for (auto it = playlist.rbegin(); it != playlist.rend(); ++it) cout << *it
<< " | ";
}

```

7. Complexity Summary

Operation	Singly (head+tail)	Doubly (sentinels)	Circular (tail ref)
Insert front / back	O(1) / O(1)	O(1) / O(1)	O(1) / O(1)
Delete front	O(1)	O(1)	O(1)
Delete back	O(n)	O(1)	O(n) (singly circular)

Operation	Singly (head+tail)	Doubly (sentinels)	Circular (tail ref)
Delete a known node	O(n)	O(1)	O(n)
Search / access by index	O(n)	O(n)	O(n)
Reverse traversal	-	O(n)	-

8. Common Mistakes

Mistake	Effect	Fix
Updating only next , forgetting prev	Backward traversal breaks	Always update both directions
Changing neighbours before setting the new node's links	Lost references	Set node.prev/next first
while cur: on a circular list	Infinite loop	Stop when back at the start node
Removing the tail of a circular list without updating tail	Ring points to a removed node	Update tail to the previous node
Printing sentinel nodes as data	None values in output	Start at head.next , stop at tail

9. Lab Tasks

Solve all tasks in `Lab09_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Insert After a Value

Add a method `insert_after(target, data)` to `DoublyLinkedList` (the notebook re-defines the class) that inserts `data` right after the first node containing `target`. Return `True` if inserted, `False` if `target` is not found.

```
[10, 20, 30].insert_after(20, 25) → [10, 20, 25, 30]
```

Task 2 Reverse a Doubly Linked List

Add `reverse()` to `DoublyLinkedList`: swap `prev` and `next` in **every** node (including the sentinels) and then swap the head and tail sentinels. Both `forward()` and `backward()` must still work.

```
[1, 2, 3] → [3, 2, 1]
```

Task 3 Browser History

Implement `BrowserHistory(homepage)` with methods `visit(url)`, `back(steps)` and `forward(steps)` using a doubly linked list of pages and a `current` reference.

`visit` clears all forward history. `back/forward` move at most as far as possible and return the current URL.

```
b = BrowserHistory("uo.edu.pk"); visit("lms"); visit("results")
back(1) → "lms"; back(5) → "uo.edu.pk"; forward(1) → "lms"
visit("timetable"); forward(2) → "timetable"
```

Task 4 Josephus Problem

n people stand in a circle numbered 1 to n . Starting from person 1, every k -th person is eliminated, and counting continues from the next person. Return the **order of elimination** and the **survivor**.

Simulate it with a circular linked list ($O(n \cdot k)$). The story: the historian Josephus used this to find the safe position.

```
josephus(7, 3) → ([3, 6, 2, 7, 5, 1], 4)
```

Task 5 🟡 Playlist with Repeat-All

Build a **Playlist** class on a **circular doubly linked list** with: `add(song)`, `next()`, `prev()` (both wrap around), `current()` and `remove_current()` (removes the playing song and moves to the next one).

Return **None** from `current()` when the playlist is empty.

```
add A, B, C → current A; prev() → C; next() → A; next() → B; remove_current()
→ now C
```

Task 6 🔴 Delete Duplicates from a Sorted DLL

Write `dedupe(dll)` that removes duplicate values from a **sorted DoublyLinkedList** in a single pass using `_unlink`, keeping the first occurrence. Make sure `backward()` still gives the correct result.

```
[1, 1, 2, 3, 3, 3, 4] → [1, 2, 3, 4]
```

Task 7 🔴 LRU Cache (Interview Classic)

Design an `LRUCache(capacity)` with `get(key)` and `put(key, value)`, **both in $O(1)$** . When the cache is full, `put` evicts the **least recently used** key. Any `get` or `put` makes a key the most recently used.

Design: a dictionary maps `key` → DLL node; the DLL keeps keys in order of use (most recent at the front). Use `DNode` with `data = (key, value)`.

```
c = LRUCache(2); put(1,1); put(2,2); get(1) → 1; put(3,3) # evicts 2
get(2) → -1; put(4,4) # evicts 1
get(1) → -1; get(3) → 3; get(4) → 4
```



10. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab09** folder.

#	Problem	Level	Concept practised
707	Design Linked List	Medium	Build a full linked list
1472	Design Browser History	Medium	Doubly linked list (Task 3)
146	LRU Cache	Medium	Hash map + DLL (Task 7)

#	Problem	Level	Concept practised
641	Design Circular Deque	Medium	Circular structure
430	Flatten a Multilevel Doubly Linked List	Medium	DLL pointer manipulation
1823	Find the Winner of the Circular Game	Medium	Josephus (Task 4)



11. Home Task

1. Finish all tasks and push `Lab09_Student.ipynb` to GitHub.
2. Add `__iter__`, `__reversed__` and `__contains__` to `DoublyLinkedList` so it works with `for`, `reversed()` and `in`.
3. Implement a **circular buffer** of fixed size for sensor readings: new readings overwrite the oldest when full. Compute the moving average of the last 10 readings.
4. **Mini-project: Text Editor Line Buffer.** Store lines of text in a doubly linked list with a cursor. Support: move up/down, insert a line, delete the current line, undo the last delete, and print the document.
5. Solve at least 3 LeetCode problems from the list.

? 12. Viva Questions

1. What are the three parts of a doubly linked list node?
2. Give two advantages and one disadvantage of a DLL compared to a singly linked list.
3. Why can a DLL delete a known node in $O(1)$?
4. What are sentinel nodes and why are they useful?
5. How many references change when inserting a node in the middle of a DLL?
6. What is a circular linked list? How do you traverse it without an infinite loop?
7. Why keep a reference to the tail instead of the head in a circular list?
8. Explain round-robin scheduling.
9. Explain the Josephus problem.
10. How does an LRU cache achieve $O(1)$ get and put?



13. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



14. References and Further Learning

- Python `collections.deque`: docs.python.org/3/library/collections.html#collections.deque
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.